

REPORTE TÉCNICO

Asunto: Sistema de Información Web **Postgrado:** Maestría en Ingeniería en Electrónica
Tarea 04: Maven, Hibernate y Spring
Tutores: Dr. Julio Mello / MSc. Marcos Benítez
Autor: Aldo Reyes Mella
Fecha: 30 de marzo del 2025
Clasificación del documento: Público



TABLA DE CONTENIDO

1. Introducción	3
2. Maven.....	4
2.1. Qué es un arquetipo y cómo utilizarlo (describir dos ejemplos de arquetipo y su uso).....	4
2.2. Cuáles son las fases que utiliza el Maven para la compilación y empaquetado de proyecto? Así también explicar la diferencia entre utilizar el objetivo (package e install)	4
2.3. Qué son los plugins en Maven para qué sirven y cómo se puede utilizar (dar dos ejemplos de plugins).....	5
3. Hibernate	6
3.1. ¿Qué es Hibernate en proyectos Java ?	6
3.2. ¿Cuál es la diferencia entre un ORM y utilizar JDBC?	7
3.3. Citar herramientas ORM que son utilizadas para proyecto java (por lo menos tres) ..	7
3.4. Realizar dos ejemplos de definición de Clase Entidad utilizando las anotaciones de Hibernate	7
4. Clonar el proyecto en la máquina local.....	10
4.1. ¿Qué es Spring y cómo ayuda al desarrollador Java?	10
4.2. ¿Qué es Spring Boot y por qué utilizarlo?	10
4.3. ¿Qué es el patrón de diseño MVC y cuales son sus componentes? (Explique cómo el Spring lo implementa)	10
4.4. ¿Qué es Spring Security y por qué es necesario utilizarlo?	11
4.5. Explique cuales son los componentes principales del Spring Security.....	11
4.6. Explique cómo el Spring Security implementa los controles de seguridad de acceso a recursos	12
5. Conclusión.....	13
6. Bibliografía	13

1. Introducción

Para crear aplicaciones Java robustas y escalables, herramientas como Maven, Hibernate y Spring son clave: Maven simplifica la gestión de proyectos, Hibernate facilita el trabajo con bases de datos y Spring agiliza el desarrollo seguro. Este informe explora sus funciones esenciales—desde arquetipos en Maven hasta anotaciones en Hibernate y la seguridad con Spring—brindando una guía clara para integrarlas en proyectos reales.



2. Maven

2.1. Qué es un arquetipo y cómo utilizarlo (describir dos ejemplos de arquetipo y su uso)

Un arquetipo en Maven es un patrón de proyecto predefinido que permite generar la estructura básica de un proyecto de software con una disposición estándar de archivos y carpetas. Los arquetipos facilitan la inicialización rápida de proyectos al proporcionar una base estructurada y coherente.

- **Ej. 1: Arquetipo maven-archetype-quickstart:** Este arquetipo genera una estructura básica para un proyecto Java con la siguiente jerarquía:

```
nombre-proyecto/
├── pom.xml
├── src/
│   ├── main/java/com/ejemplo/App.java
│   └── test/java/com/ejemplo/AppTest.java
```

```
mvn archetype:generate -DgroupId=com.ejemplo \
-DartifactId=nombre-proyecto \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

- **Ejemplo 2: Arquetipo maven-archetype-webapp:** Este arquetipo crea un proyecto web con soporte para Servlets y JSP:

```
mvn archetype:generate -DgroupId=com.webapp \
-DartifactId=mi-webapp \
-DarchetypeArtifactId=maven-archetype-webapp \
-DinteractiveMode=false
```

La estructura generada incluye una carpeta webapp/ con WEB-INF/ y un archivo index.jsp.

2.2. Cuáles son las fases que utiliza el Maven para la compilación y empaquetado de proyecto? Así también explicar la diferencia entre utilizar el objetivo (package e install)

Maven organiza el ciclo de vida de construcción del proyecto en fases. Algunas de las más importantes son:

- **Principales fases del ciclo de vida**

<i>Fase</i>	<i>Descripción</i>
validate	Verifica que la estructura del proyecto sea correcta.
compile	Compila el código fuente.
test	Ejecuta pruebas unitarias con frameworks como JUnit.
package	Genera un archivo .jar o .war con el código compilado.
verify	Ejecuta pruebas de integración y validación.
install	Instala el paquete en el repositorio local para su uso en otros proyectos.
deploy	Publica el paquete en un repositorio remoto.

- **Diferencia entre package e install**

- ✓ **mvn package:** Genera el archivo empaquetado (.jar o .war), pero no lo almacena en el repositorio local.
- ✓ **mvn install:** Genera el paquete y lo instala en el repositorio local (~/.m2/repository/), permitiendo su uso por otros proyectos.

2.3. Qué son los plugins en Maven para qué sirven y cómo se puede utilizar (dar dos ejemplos de plugins)

Los plugins en Maven son componentes que extienden sus funcionalidades, permitiendo tareas adicionales como compilación, pruebas, generación de documentación y despliegue.

Ej. 1: Plugin maven-compiler-plugin: Este plugin se utiliza para compilar código fuente Java especificando la versión del JDK:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
      <configuration>
        <source>11</source>
        <target>11</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Ej. 2: Plugin maven-surefire-plugin: Este plugin permite la ejecución de pruebas unitarias con JUnit o TestNG:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.0.0-M7</version>
    </plugin>
  </plugins>
</build>
```

Para ejecutar las pruebas unitarias:

```
mvn test
```

3. Hibernate

3.1. ¿Qué es Hibernate en proyectos Java ?

Hibernate es un framework de **mapeo objeto-relacional (ORM, por sus siglas en inglés)** que facilita la interacción entre las aplicaciones Java y las bases de datos relacionales. Se encarga de la conversión automática de objetos Java en registros de base de datos y viceversa, eliminando la necesidad de escribir sentencias SQL manualmente.

Algunas características clave de Hibernate incluyen:

- **Independencia de base de datos:** Permite cambiar de base de datos sin modificar el código de la aplicación.
- **Reducción del código SQL:** Minimiza la cantidad de código SQL escrito manualmente.
- **Gestión de transacciones:** Simplifica la administración de transacciones y sesiones.
- **Soporte para consultas avanzadas:** Hibernate Query Language (HQL) y Criteria API.

3.2. ¿Cuál es la diferencia entre un ORM y utilizar JDBC?

CARACTERÍSTICA	JDBC	ORM (HIBERNATE)
MANEJO DE SQL	Se deben escribir consultas SQL manualmente.	Genera consultas SQL automáticamente.
MANTENIMIENTO	Difícil de mantener si hay cambios en la base de datos.	Flexible ante cambios en la base de datos.
OPTIMIZACIÓN	Necesita optimización manual de consultas.	Ofrece técnicas como caché y lazy loading.
GESTIÓN DE TRANSACCIONES	Requiere manejo manual de commits y rollbacks.	Proporciona una gestión automática de transacciones.

3.3. Citar herramientas ORM que son utilizadas para proyecto java (por lo menos tres)

Existen varias herramientas ORM utilizadas en proyectos Java. Algunas de las más populares son:

- **Hibernate:** La más utilizada en el ecosistema Java, proporciona una abstracción robusta sobre JDBC y soporte para diferentes bases de datos.
- **EclipseLink:** Implementación de JPA (Java Persistence API) de código abierto mantenida por Eclipse Foundation.
- **MyBatis:** Un framework ORM que permite el mapeo SQL con menos abstracción que Hibernate, brindando mayor control sobre las consultas.

3.4. Realizar dos ejemplos de definición de Clase Entidad utilizando las anotaciones de Hibernate

A continuación, se presentan dos ejemplos de clases entidad en Hibernate:

Ej. 1: Clase "Usuario"

```
import jakarta.persistence.*;

@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", nullable = false, length = 100)
    private String nombre;

    @Column(name = "email", unique = true, nullable = false)
    private String email;

    // Getters y Setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```




Ej. 2: Clase "Producto" con Relación a "Categoría"

```
import jakarta.persistence.*;
@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", nullable = false)
    private String nombre;

    @Column(name = "precio")
    private Double precio;

    @ManyToOne
    @JoinColumn(name = "categoria_id", nullable = false)
    private Categoria categoria;

    // Getters y Setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public Double getPrecio() {
        return precio;
    }
    public void setPrecio(Double precio) {
        this.precio = precio;
    }
    public Categoria getCategoria() {
        return categoria;
    }
    public void setCategoria(Categoria categoria) {
        this.categoria = categoria;
    }
}
```

4. Clonar el proyecto en la máquina local

4.1. ¿Qué es Spring y cómo ayuda al desarrollador Java?

Spring es un framework de desarrollo para Java que proporciona una infraestructura integral para la creación de aplicaciones robustas y flexibles. Su objetivo principal es facilitar la implementación de aplicaciones basadas en Java reduciendo la complejidad inherente al desarrollo empresarial. Spring ayuda a los desarrolladores mediante:

- **Inversión de control (IoC):** Gestiona la creación y configuración de objetos en la aplicación.
- **Inyección de dependencias (DI):** Facilita el mantenimiento y la prueba de aplicaciones.
- **Programación orientada a aspectos (AOP):** Permite la separación de preocupaciones transversales como seguridad y transacciones.
- **Soporte para diversos modelos de programación:** Como JDBC, ORM, JMS y MVC.

4.2. ¿Qué es Spring Boot y por qué utilizarlo?

Spring Boot es una extensión del framework Spring que facilita la creación de aplicaciones con una configuración mínima. Sus principales beneficios incluyen:

- **Autoconfiguración:** Reduce la necesidad de configurar manualmente componentes.
- **Servidor embebido:** Permite ejecutar aplicaciones sin necesidad de servidores externos.
- **Convención sobre configuración:** Facilita el desarrollo rápido.
- **Integración con Spring Security, JPA, y otros módulos.**

Ejemplo de creación de una aplicación Spring Boot:

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

4.3. ¿Qué es el patrón de diseño MVC y cuales son sus componentes? (Explique cómo el Spring lo implementa)

El patrón **Modelo-Vista-Controlador (MVC)** es un enfoque arquitectónico que separa la lógica de negocio, la presentación y la gestión de eventos en una aplicación.

- **Modelo:** Gestiona los datos y la lógica de negocio.
- **Vista:** Muestra la información al usuario.
- **Controlador:** Maneja la interacción del usuario y actualiza el modelo y la vista.

Spring implementa MVC a través del módulo spring-web con `@Controller` y `@RestController`:

```
@Controller
@RequestMapping("/productos")
public class ProductoController {
    @GetMapping("/")
    public String listarProductos(Model model) {
        model.addAttribute("productos", servicio.obtenerProductos());
        return "productos";
    }
}
```

4.4. ¿Qué es Spring Security y por qué es necesario utilizarlo?

Spring Security es un framework que proporciona autenticación y autorización para aplicaciones basadas en Spring. Su importancia radica en:

- **Protección contra ataques comunes** (CSRF, XSS, inyección SQL, etc.).
- **Gestor de autenticación robusto**, compatible con OAuth2, JWT, LDAP, entre otros.
- **Control de acceso basado en roles y reglas de negocio.**

Ejemplo de configuración básica en Spring Security:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin").hasRole("ADMIN")
            .anyRequest().authenticated()
            .formLogin(withDefaults());
        return http.build();
    }
}
```

4.5. Explique cuales son los componentes principales del Spring Security

Spring Security cuenta con los siguientes componentes clave:

- **AuthenticationManager:** Gestiona la autenticación de usuarios.
- **SecurityContextHolder:** Almacena los detalles de autenticación.
- **UserDetailsService:** Recupera los detalles del usuario.
- **GrantedAuthority:** Representa los permisos de un usuario.
- **Security Filters:** Interceptan peticiones HTTP para verificar la seguridad.
- **AccessDecisionManager:** Determina si un usuario puede acceder a un recurso.

4.6. Explique cómo el Spring Security implementa los controles de seguridad de acceso a recursos

Spring Security permite restringir el acceso a recursos mediante reglas declarativas y programáticas.

- **Restricción por anotaciones:**

```
@PreAuthorize("hasRole('ADMIN')")
public void eliminarUsuario(Long id) {
    usuarioRepo.deleteById(id);
}
```

- **Configuración basada en URL:**

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .anyRequest().permitAll());
```

- **Uso de JWT para autenticación sin estado:**

```
public String generateToken(UserDetails userDetails) {
    return Jwts.builder()
        .setSubject(userDetails.getUsername())
        .signWith(SignatureAlgorithm.HS512, SECRET_KEY)
        .compact();
}
```

5. Conclusión

Maven, Hibernate y Spring forman un equipo poderoso para crear aplicaciones Java robustas y escalables. Maven simplifica la gestión del proyecto, Hibernate hace que trabajar con bases de datos sea más intuitivo, y Spring (junto con Spring Boot y Spring Security) ofrece herramientas completas para desarrollar rápido y con seguridad. Dominar estas tecnologías permite a los desarrolladores construir software eficiente, mantenible y alineado con los estándares actuales, haciendo que el proceso sea más ágil y los resultados, más confiables.

6. Bibliografía

- Apache Maven. (2025). "Introduction to Maven." Disponible en: <https://maven.apache.org/>
- Hibernate ORM. (2025). "What is Hibernate?" Disponible en: <https://hibernate.org/>
- Spring Framework. (2025). "Spring Framework Documentation." Disponible en: <https://spring.io/>
- Richardson, C. (2019). "Microservices Patterns: With examples in Java." Manning Publications.
- Fowler, M. (2002). "Patterns of Enterprise Application Architecture." Addison-Wesley.

Reporte: https://github.com/ajreymel/Sistemas-de-Informaci-n-Web/tree/nueva_rama/Semana03